

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – AUTUMN 2026

Final Project Report

Topic 4 – Async Research Assistant

Project	Software_Final-Project_M201
Report date	19 September 2026
Repository	https://github.com/Dilara898/Software_Final-Project_M201
Contributors	Dilara Rzazada; Melek Askerova; Sanan Hajiyev; Nezrin Ibrahimova
Code reviewed	7c2d996e8a748f6d0515f331df82d932120bd982
Release status	Reviewed development snapshot; v1.0-final not present in fetched tags.

Executive Summary

We built an Async Research Assistant that accepts a natural-language question, collects excerpts from Wikipedia, arXiv and Tavily, and uses Gemini to produce an answer with numbered references. The software-engineering layer adds bounded asynchronous collection, PostgreSQL-backed source caching and history, retries, validation, structured diagnostics, a CLI and a Streamlit interface around the unchanged course-provided AI package. In the latest three-repeat offline benchmark over five questions, median sequential collection took 1133.230 ms and concurrent collection took 435.011 ms: a $2.605\times$ speedup. A fresh test run on main 7c2d996 passed 350 tests and reported 97% combined statement-and-branch coverage of researcher/. Ruff, mypy and dependency checks passed. Historical live runs demonstrate partial-source answers and saved history, but also provider and quota failures. Explicit boundaries and source isolation make the workflow easier to test and diagnose. If starting again, we would introduce shared provider-budget management and reproducible performance capture earlier.

Evidence boundary. Tests were rerun on Windows/Python 3.12.14 on 20 September. The team-supplied benchmark used Windows/Python 3.14.4 at 06:20:42 UTC, Git 7c2d996, dirty=False. CPU/RAM details were not supplied. The speedup measures simulated collection, not live end-to-end performance.

Contents

Executive Summary	1
1 Project Overview	2
2 Architecture	3
3 Concurrency & Performance	4
4 Robustness & Error Handling	5
5 Storage & Caching	6
6 Testing	7
7 Deployment & Reproducibility	8
8 Limitations & Future Work	9
9 Tools & Acknowledgements	9
References	10
A Reproducing the benchmark	10

1. Project Overview

1.1 Problem framing & scope

Research questions often require background material, academic papers and current web information. Retrieving those sources one after another introduces avoidable waiting and makes a slow provider dominate the experience. Our application takes a question and an optional source subset, retrieves candidate excerpts concurrently, and returns a synthesized answer with bracketed citations, reference titles and URLs. It also reports failed or empty providers so that a partial answer is not presented as a complete search.

The required interface is `python-mresearcherask"..."; --sourceswiki,arxiv` restricts collection and `--no-cache` bypasses source-cache reads and writes. The `history--limitN` command retrieves saved sessions. A bonus Streamlit interface, launched with `streamlitrunresearcher/ui.py`, offers Ask and History views. Both front ends share `application.build_result`; neither uses a separate HTTP API. Topic 4 does not require an HTTP server, image processing, embeddings or model training, so these are outside the implementation scope.

We tightened the basic specification by checking citation-to-source mappings, separating empty from failed sources, preserving cancellation, and distinguishing a history write that failed from one whose commit status is unknown. We did not treat valid citations as proof of factual correctness. The mandatory real-workload sequential/concurrent comparison remains unverified: the measured benchmark here is explicitly simulated, and the live provider incidents remain part of the limitations.

1.2 Provider choice and the AI module contract

The selected stack is **Gemini for synthesis, Tavily for web search, Wikipedia for encyclopedic content and arXiv for papers**. Gemini was available to the team and had previously produced successful cited answers; Tavily supplies the **Source** records expected by the provided fetcher. Provider quotas and access remain operational dependencies rather than guarantees of the application.

The wrappers call `ai.sources.fetch_wikipedia`, `fetch_arxiv`, `fetch_web` and `ai.synthesizer.synthesize`. A byte comparison of the reviewed repository's `ai/` Python files against the original SWENG Topic 4 package found no changes. **We preserve the provided AI package and its public interface.** Provider-specific transport handling is implemented outside that boundary.

Model evidence	What can be established
Historical successful artifacts	The saved 19 September web-only run records gemini-3.6-flash .
Current example configuration	<code>LLM_MODEL=</code> is blank. The CLI/UI remove the blank value; the supplied Gemini adapter then defaults to gemini-2.0-flash .
Latest local measurements	Offline tests/benchmark do not establish which live model was selected. The 429 console log does not include its model ID.

The report therefore does not assign one model ID to every run. A final live demonstration should pin a model available to the team's account and save its ID in the run manifest. Switching to an alternative supplied provider requires the corresponding environment values, credentials and SDK dependency; it is not a claim that every alternative SDK is installed in the current image.

Before an answer is persisted, `validate_answer` checks a nonempty answer, the original question, matching marker/reference sets, unique citation indices, in-range indices and equality with the indexed source. This rejects structural misattribution while leaving semantic truth and claim-by-claim evidential support as explicit limitations.

2. Architecture

2.1 Module map

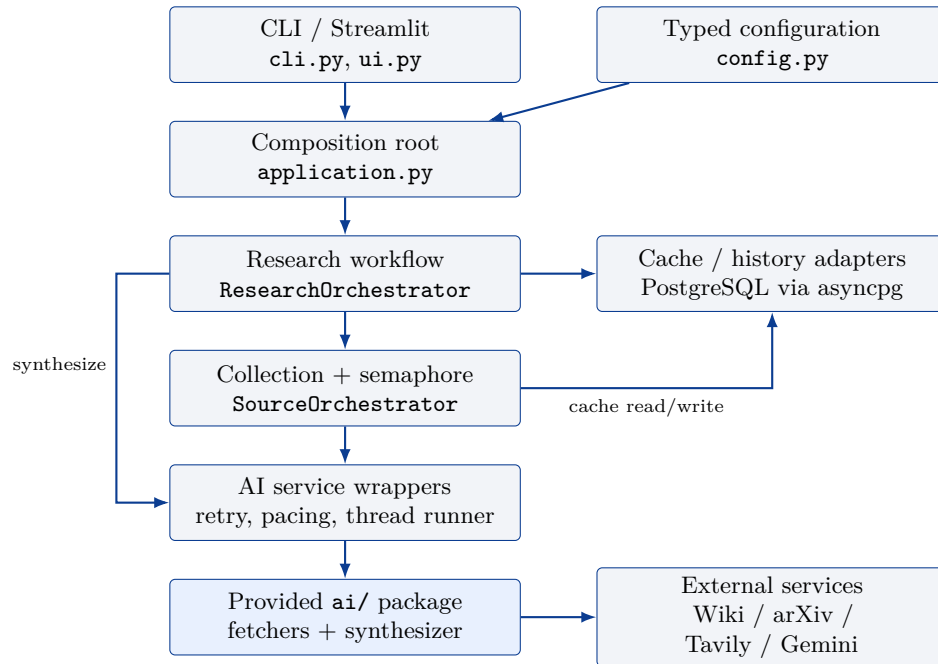


Figure 1. Logical dependency and call boundaries; results return to their callers.

`application.py` owns resources and constructs the real workflow. `concurrency/research.py` manages collect–synthesize–validate–persist; `orchestrator.py` isolates source failures and maintains cache/timing outcomes. `services/ai_service.py` crosses the AI boundary, while `http_client.py` centralizes transport policy. `storage/` owns SQL, caching and pool lifecycle; `integration.py` maps typed workflow records into storage calls. Pure formatting and source aliasing live in `core/logic.py`.

2.2 OOP design & key abstractions

```

1 class CacheStore(ABC):
2     @abstractmethod
3     async def get(self, key: str) -> list[Source] | None: ...
4     @abstractmethod
5     async def set(self, key: str, sources: list[Source]) -> None: ...
6
7 class InMemoryCacheStore(CacheStore): # implementation omitted
8     ...
9 class PostgresCacheStore(CacheStore): # implementation omitted
10    ...

```

The ABC makes the two-method, stateful cache contract explicit and prevents instantiation of incomplete subclasses. Workflow ports use `Protocol` for structural typing, allowing test doubles without inherited implementation. Composition is used for the workflow: it *has* a collector, synthesis service and history store; it is not a specialized database or provider.

2.3 Data flow across module boundaries

We reuse `Source`, `Citation` and `AnswerWithCitations` from `ai/`. Our Pydantic models add execution context: `SourceOutcome`, `CollectionResult`, `ResearchResult`, `SessionRecord`, `ResearchTimings`, `CacheStats` and `OperationWarning`. Core workflow boundaries pass these models or typed source lists. Dictionaries still exist at JSON, SDK and script-metadata edges; an absolute “no dictionaries anywhere” claim would be inaccurate. CLI arguments also remain an `argparse` namespace, with incomplete public annotations noted in Section 8.

3. Concurrency & Performance

3.1 Why this concurrency model

Source retrieval is network I/O rather than CPU-heavy computation. `asyncio.gather` overlaps waiting through one shared `httpx.AsyncClient`; source tasks have separate deadlines and a shared semaphore within the collector. The application selects **three permits**, enough to query all three sources for one question. Threads would also overlap I/O but add another concurrency mechanism around already asynchronous fetchers. Multiprocessing would add serialization/process overhead without addressing the network bottleneck. Python documents cancellation and task-lifetime responsibilities for these primitives [3]; HTTPX clients provide connection pooling [4].

The synchronous LLM call is offloaded to a dedicated `ThreadPoolExecutor`, bounded to one worker by default. An async timeout limits the caller’s wait but cannot kill an already running SDK thread. With one source permit, collection becomes effectively serial; a bound of 100 does not help a three-source question and permits larger bursts under shared load. The current application creates a collector per request, so the bound is not a process-wide or multi-process provider quota.

3.2 Sequential-vs-concurrent benchmark

Workload: five questions, three fetches per question; 15 simulated source operations per mode per repeat, 90 across both modes and three repeats. Questions remain in the same order. Caches are bypassed; one client is shared per batch. Only successful pairs with matching result counts enter the comparison. Synthesis, history, and client setup/teardown are excluded from collection timing. No warm-up was enabled.

Repeat	Sequential (ms)	Concurrent (ms)	Ratio	Order
1	947.757	441.650	2.146×	S then C
2	1189.837	435.011	2.735×	C then S
3	1133.230	428.534	2.644×	S then C
Median	1133.230	435.011	2.605 ×	

$$S = \frac{\text{median}(T_{\text{seq}})}{\text{median}(T_{\text{con}})} = \frac{1133.230}{435.011} = 2.605$$

Median collection time decreased by 61.61%. The source double sleeps for 40 ms (Wikipedia), 80 ms (arXiv) and 40 ms (web), so the ideal per-question sleep-only ratio is 2. The measured ratio also reflects scheduling and timer overhead; it does not establish better-than-ideal network scaling. Median successful parallel fetches were 44.188 ms, 84.552 ms and 43.071 ms respectively. Live performance can instead be dominated by network latency, quota or LLM synthesis.

Environment: Windows, Python 3.14.4; 20 September 2026, 06:20:42 UTC. The supplied benchmark records Git 7c2d996, dirty=False. CPU/RAM details were not supplied. All three repeats were comparable. A break-even workload and live speedup were not measured.

3.3 Rate limit & politeness

The production fetch wrapper uses a one-second minimum interval per source instance, including retries. This is a pacing policy, not a global token bucket. Transient 429/5xx and transport errors receive bounded retries with exponential jitter; identifiable permanent 4xx are not retried. The ordinary fan-out is at most three source tasks per collector; the tests exercise shared-collector bounds, not a measured live burst budget. LLM project-level RPM/TPM accounting is still absent, and the observed 429 incident demonstrates that retry alone does not ensure quota compliance.

4. Robustness & Error Handling

4.1 Wrapping the AI module

AIFetchService validates returned lists and **Source** elements and wraps fetch calls with retry, pacing and per-attempt timeouts. With the default 10-second source budget, application wiring sets a 2.5-second fetch-attempt deadline, initial exponential-wait parameter 0.25 seconds, maximum wait 0.5 seconds and up to three attempts. The outer source deadline bounds the whole operation. Wikipedia also has transport-level retry because the supplied fetcher can skip failed summary requests internally.

AISynthesisService uses up to three attempts, a 15-second default attempt timeout, initial wait parameter 0.5 seconds, maximum wait six seconds and one worker thread. The composition sets a 55-second synthesis deadline and a total research deadline of source timeout plus 65 seconds (75 seconds by default). These nested budgets must be changed together. Optional fallback factories exist, but the default application constructs the service without a fallback chain; automatic failover is not claimed as an enabled feature.

Structured INFO events include source, operation, attempt, result count, duration and request ID where propagated. WARNING records identify retry waits and unavailable sources; error output avoids raw provider exceptions that may contain secrets. The final result distinguishes unavailable and empty sources, and history states **saved**, **failed** and **unknown**. Full sanitized request/response logging at DEBUG and complete request-ID propagation are not yet implemented.

4.2 Failure-mode analysis: live provider rejection

Incident, 19 September. This was an observed live failure, not an injected one. In a single-question run, arXiv returned 406, Wikipedia produced zero results and Tavily returned three sources. Synthesis succeeded and the CLI reported **History:saved**, session 18, with a total time of 10.48 seconds. The expected partial-source behavior worked: the user received a useful answer and an explicit notice about missing providers.

A later five-question demo had the same source pattern, but Gemini returned 429 on every synthesis attempt. Each question used three application-level attempts; all five ended in **SynthesisError**, producing **Completed: 0/5**. The ten retry waits were only about 0.51–1.90 seconds. No exact quota dimension or model ID was present in that console output, so it does not establish whether a per-minute, daily or another limit was exhausted. Google documents model/project-specific limits and exponential backoff [5].

```
fetch_exhausted: source='arxiv', error_code='provider_error_status_406'
source_completed: source='wikipedia', status='empty', result_count=0
fetch_succeeded: source='web', result_count=3
provider_call_retry: attempt=1, error_code='provider_error_status_429'
llm_provider_exhausted: error_code='provider_error_status_429'
Demo question failed (SynthesisError)
```

The code centralizes HTTP client construction, upgrades the arXiv HTTP endpoint to HTTPS, follows redirects and sets host-specific Accept headers. These changes prevent transport-policy drift; offline tests do not establish live recovery. Wikipedia now logs suspicious empty results separately. Later deployment logs include successful requests, but no new five-question live benchmark is claimed. Quota-aware admission, server-directed retry delays and a saved live rerun remain necessary.

4.3 Input validation

CLI and UI reject empty questions, normalize whitespace and enforce 3–500 characters; history limits are 1–100. Source selection uses **wiki**, **arxiv**, **web** with duplicate removal and rejection of empty/unknown entries. Demo input must be a JSON object with a nonempty question list, unique filename-safe IDs and valid text. Terminal control sequences are removed from CLI answers. Typed settings reject type mismatches and missing database configuration, but strict DSN/provider/range validation is incomplete. There is no arbitrary file-upload endpoint; file-size limits and comprehensive hostile-input testing remain future work.

5. Storage & Caching

5.1 Schema

The implementation uses **PostgreSQL through asyncpg**, not the SQLite example in the report template. The two independent tables separate authoritative session history from replaceable source-cache data. The schema below is taken from `researcher/storage/schema.sql`.

```
CREATE TABLE IF NOT EXISTS cache_entries (
    key TEXT PRIMARY KEY,
    payload JSONB NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_cache_expires
    ON cache_entries (expires_at);

CREATE TABLE IF NOT EXISTS research_sessions (
    id SERIAL PRIMARY KEY,
    question TEXT NOT NULL,
    answer TEXT NOT NULL,
    sources_used TEXT[] NOT NULL DEFAULT '{}',
    sources_failed TEXT[] NOT NULL DEFAULT '{}',
    duration_ms INTEGER NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The cache primary key supports point lookup and upsert; the expiry index supports expiration-oriented maintenance. History has its primary-key index but no dedicated `created_at` index, although recent-session reads sort by that field. There are no foreign keys because cache entries and sessions are not linked relationally. Adding an ordering index would be useful at large history volumes.

The session stores question, answer, source-provider lists and answer-ready duration. It does *not* store complete citation objects, source excerpts or the selected model. Consequently, history cannot independently reconstruct the full reference list or the original provider context. The answer text is a record of the generated response, not an independently verified factual source of truth. A future schema should preserve immutable citation snapshots and run/model metadata.

Pool construction is lazy and guarded by a lock; the configured pool uses one to five connections. The application owns creation and shutdown. Storage adapters translate expected connection failures into workflow warnings. A history deadline can leave commit status unknown: automatically replaying the insert might duplicate a session, so the renderer warns against automatic retry.

5.2 Caching policy

A cache key combines the source name with a SHA-256 digest of the normalized question. Normalization lowercases text, collapses whitespace and strips trailing question/exclamation/full-stop punctuation. `PostgresCacheStore` serializes source models as JSON and writes them into JSONB with an expiry timestamp. JSONB is suitable for variable-length source records; there are no embeddings or vector dimensions in this topic.

The default TTL is **86,400 seconds (24 hours)**. Expired rows are treated as misses and deleted when accessed. A successful upsert replaces both payload and expiry. `--no-cache` bypasses reads and writes; empty or failed collections are not cached by the orchestrator. The in-memory implementation is useful for tests; PostgreSQL permits persistence across CLI processes and UI sessions.

The application now allows 3 seconds per cache read/write via `CACHE_TIMEOUT_SECONDS` (greater than 0, at most 10); the standalone collector retains 0.5 seconds. Cache failures fall back to fresh fetching and remain logged. The UI shows a cache-issue count instead of internal warning boxes. A startup DB failure still blocks research. Cached sources do not cache LLM answers or remove Gemini quota usage. No measured aggregate live cache-hit rate is claimed.

Invalidation limitation. Keys omit web-provider choice and maximum-result count. Changing these settings may reuse stale or differently sized entries until expiry. A versioned key including result-affecting configuration, plus scheduled expiry cleanup, would make cache behavior more reproducible.

6. Testing

6.1 Strategy & coverage

A fresh run on Windows/Python 3.12.14, main 7c2d996, passed 350 tests in 22.36 seconds. The scope is researcher/ with branch measurement enabled: 1,356 statements, 25 missed statements, 298 branches and 19 partial branches; combined coverage rounds to 97%. The provided ai/ package is excluded from the coverage denominator, while its smoke tests remain in the suite. Coverage measures execution, not factual answer accuracy or live service health [6].

```
python -m pytest tests -q --cov=researcher --cov-branch --cov-report=term-missing
# Fresh local run: 350 passed in 22.36s
```

The command used for the measurement does not enforce a threshold. For a release gate, append `--cov-fail-under=60`. The earlier 280-test result measured **researcher** and **ai** without branch coverage and reported 77.72%; it is not directly comparable with 97%. New tests and changed measurement scope both matter.

Module	Coverage	Module	Coverage
application	100%	CLI	98%
research workflow	100%	orchestrator	99%
benchmarking	99%	integration	95%
AI service	97%	retry	97%
HTTP client	100%	transport	100%
cache store	94%	database pool	96%
history	100%	validation	100%
Streamlit UI	89%	entry point	80%

The suite uses fake providers, AsyncMock and HTTPX MockTransport, including the original 16 smoke-test functions. It checks behavior without paid provider requests. The supplied ai/ package remains outside the engineering changes. An earlier separately audited run blocked network access and passed 280 tests; the current ordinary pytest run does not constitute a new network-isolation audit.

6.2 Notable tests

1. `test_complete_research_has_typed_result_and_one_history_write` checks the successful workflow with injected ports: a typed answer and one history write.
2. `test_source_timeout_preserves_fast_results_and_cancels_slow_task` verifies that a slow source does not discard fast siblings. `test_semaphore_is_shared_across_simultaneous_collections` checks the collector-level bound.
3. `test_partial_and_all_source_failures` distinguishes partial success from no usable sources, rather than sending an empty evidence set to synthesis.
4. `test_invalid_citations_are_rejected` and grouped-citation tests reject mismatched markers, indices and source mappings.
5. `test_history_timeout_is_bounded` and history-failure tests preserve a usable answer while avoiding unsafe automatic replay.
6. UI tests cover validation, rendering, unavailable sources, history and cache-warning filtering. Integration tests also cover cache reads exceeding the collector's 0.5-second default.

Fresh checks on 7c2d996 passed: Ruff for researcher, tests and scripts; mypy for 28 source files; and pip check. The full suite includes the merged cache-deadline and UI-warning regression tests. Pinned requirements include Streamlit 1.64.0. Public type annotations remain incomplete in some composition/storage methods despite the passing permissive mypy configuration. No fresh Docker build or paid live-provider test is claimed.

7. Deployment & Reproducibility

7.1 Docker image

The Dockerfile is **single-stage**, based on `python:3.12-slim`, with pinned top-level requirements and `pipcheck`. It copies the application, supplied AI package, data and scripts, creates a non-root `appuser`, and sets `python-mresearcher` as the entrypoint. The team supplied a successful cached build lasting approximately 4.1 seconds. The image size was not measured; no size or multi-stage bonus is claimed. Slim is a reasonable balance between a small base and conventional Linux binary dependencies; transitive dependencies and the base image digest are not locked by the source files.

Compose starts PostgreSQL 16, waits for its health check, mounts `schema.sql` for first-time DB initialization and overrides the application DSN to use service hostname `db`. A named volume preserves the database. Schema initialization does not automatically migrate an existing volume after schema changes. The standard image targets the CLI; tests are not copied into it, and publishing the Streamlit port requires an explicit UI launch configuration.

```
docker build -t researcher .
docker compose up -d db
docker compose build app
docker compose run --rm app ask "What is photosynthesis?" --no-cache
docker compose run --rm app history --limit 5
# Standalone image, with an accessible PostgreSQL DSN in its environment:
docker run --rm --env-file .env researcher ask "What is photosynthesis?"
```

Standalone Docker on Windows requires a reachable host DB name such as `host.docker.internal`. Rebuild images after source updates; the older successful build does not verify this revision. Streamlit file watching is now disabled for deployment: reboot the app after updates. Local development can enable `--server.fileWatcherType=auto`.

7.2 Configuration

Variable	Default / missing-value behavior
LLM_PROVIDER	CLI/UI default to <code>gemini</code> ; supplied factories also support other providers.
LLM_MODEL	Blank is removed by entry points; provider-specific default then applies. Pin for a live release.
GOOGLE_API_KEY	Required for Gemini; adapter also accepts <code>GEMINI_API_KEY</code> or <code>LLM_API_KEY</code> . Absence raises a provider error.
WEB_SEARCH_PROVIDER	CLI/UI default to <code>tavily</code> ; other supplied choices require appropriate setup.
TAVILY_API_KEY	Required for selected Tavily search; no successful search is assumed if missing.
DATABASE_URL	Required by settings; Compose supplies its internal PostgreSQL URL.
LOG_LEVEL	INFO; logging resolves unrecognized names to an INFO fallback.
CACHE_TTL_SECONDS	86400; typed as integer, without a strict positive-range constraint in Settings.
PER_SOURCE_TIMEOUT_SECONDS	10; outer source budget in seconds.
MAX_SOURCES_PER_QUERY	3; despite its name, used as maximum results per selected source.
POSTGRES_HOST_PORT	Compose host port defaults to 5432.

`CACHE_TIMEOUT_SECONDS` now defaults to 3 seconds per cache operation, constrained to greater than 0 and at most 10. Other numeric/DSN validation remains incomplete. Alternative providers require their own keys and SDKs. Streamlit secrets are bridged to environment values; `.env` does not override existing values. Credentials stay outside the repository. Lazy settings allow imports and CLI help without a database.

8. Limitations & Future Work

1. **Live integration remains conditional.** The recorded five-question run failed with Gemini 429, arXiv returned 406 and Wikipedia returned empty results. Later offline success does not close these incidents. The next step is targeted provider diagnosis, followed by a new saved live run with model and quota context.
2. The performance claim is deliberately narrow. The $2.605\times$ figure measures simulated collection on clean Git revision 7c2d996. A real-workload comparison, hardware metadata and end-to-end measurements remain necessary. Break-even workload size and sustained-load behavior have not been established.
3. **Rate control and cancellation have limits.** Request-local semaphores and pacing are not shared account quotas. A timed-out SDK thread may continue running. Add provider-level RPM/TPM admission, server-directed retry delays and SDK timeouts; do not merely increase retry counts during exhaustion.
4. **Persistence and validation need strengthening.** Persist citation/model snapshots; version cache keys; index recent history; validate DSNs and numeric ranges at startup. Complete public type annotations and eliminate duplicated resource-lifecycle wiring where safe. The main design weakness was initially allowing similar client/resource setup paths to drift between entry points.
5. **Quality, security and release evidence are incomplete.** Citation mapping does not guarantee factual support, nor is there a full prompt-injection defense for external text. Authentication, multi-user isolation, load testing, a measured final image size and a clean final tag are outside the verified evidence. These should precede any claim of production readiness.

Team contributions and Git workflow

The following approximate shares reflect the team's agreed overall contribution, including implementation, testing and documentation, rather than the number of Git commits.

Member	Approx.	Key deliverables
Dilara Rzazada	25%	Configuration and data models; PostgreSQL setup, connection pool, cache and history storage; storage tests; final report preparation.
Melek Askerova	25%	AI fetch and synthesis services; retry and pacing mechanisms; structured logging; Streamlit UI; service and UI tests; README documentation.
Sanan Hajiyeve	25%	Concurrency and research orchestration; storage adapters; benchmark scripts; cancellation tests; shared HTTP client and transport fixes; contribution statement preparation.
Nezrin Ibrahimova	25%	CLI commands and input validation; application and Docker/Compose integration; demo scripts; CLI and integration tests; presentation preparation.

Table 1. Team contribution summary.

Main 7c2d996 includes the deployment, cache-deadline and UI-warning fixes in PRs #31 and #32. The benchmark records a clean tree and the fresh tests use this revision. Review approvals and branch protection were not audited. No v1.0-final remote tag was returned on 20 September; release tagging and the signed contribution statement remain team actions.

9. Tools & Acknowledgements

AI Academy supplied the original course ai/ package, Topic 4 data, smoke tests and report template. ChatGPT/Codex supported development and documentation through technical explanations, code review assistance, troubleshooting, interpretation of test results, and report drafting and editing. The reported results are based on the project repository and recorded test and benchmark outputs.

References

- [1] AI Academy, National AI Center. *Software Engineering Final Project Brief*, version 2.0; Topic 4 TOPIC.md; REPORT_TEMPLATE.tex. Supplied SWENG archive.
- [2] Project source, revision 7c2d996. https://github.com/Dilara898/Software_Final-Project_M201. Reviewed 20 September 2026.
- [3] Python documentation. *Coroutines and Tasks*. <https://docs.python.org/3/library/asyncio-task.html>.
- [4] HTTPX documentation. *Clients*. <https://www.python-httpx.org/advanced/clients/>.
- [5] Google AI for Developers. *Gemini API rate limits and troubleshooting*. <https://ai.google.dev/gemini-api/docs/rate-limits>; <https://ai.google.dev/gemini-api/docs/troubleshooting>.
- [6] Coverage.py documentation. *Branch coverage measurement*. <https://coverage.readthedocs.io/en/latest/branch.html>.
- [7] Docker documentation. *docker compose run*. <https://docs.docker.com/reference/cli/docker/compose/run/>.
- [8] Evidence: fresh 20 September tests on 7c2d996 (350 passed, Windows/Python 3.12.14); team-supplied offline benchmark at 06:20:42 UTC (Python 3.14.4, dirty=False); historical 19 September build and live logs. Benchmark outputs: benchmark-offline-final.md and benchmark-offline-final.csv.

A. Reproducing the benchmark

These PowerShell commands check out the exact revision recorded by the supplied benchmark. Use Python 3.14.4 for its runtime; the fresh test run used Python 3.12.14. Timing variance is expected. Hardware details were not supplied, and no guaranteed timing tolerance is claimed.

```
git clone https://github.com/Dilara898/Software_Final-Project_M201.git
cd Software_Final-Project_M201
git checkout 7c2d996e8a748f6d0515f331df82d932120bd982
python -m venv .venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
.\venv\Scripts\python.exe scripts/benchmark.py --offline --repeats 3 --out artefacts/benchmark-offline-final.md --csv
  artefacts/benchmark-offline-final.csv
.\venv\Scripts\python.exe -m pytest tests -q --cov=researcher --cov-branch --cov-report=term-missing --cov-fail-under=60
```

The dataset is data/research_questions.json; SHA-256:

f164885b4bcb564f4d473b495540718485e91f0d7ed562461ddd7dd77e443d2f

The offline factory is scripts.c_offline:OfflineService. Setup ranged from 0.134–0.209 ms and teardown from 0.006–0.011 ms, excluded from collection medians. Raw CSV timing columns repeat per source row and must not be summed as independent batch times.

Benchmark evidence. Keep the Markdown summary and raw CSV together with the test output, machine information and the exact Git revision. Setup and teardown should remain separate from collection time, and offline measurements should retain their simulation label. These records make the reported comparison traceable and allow later measurements to be compared on the same basis.